

TL;DR: операционные компоненты держат систему живой в продакшене

Table of contents

TL;DR: операционные компоненты держат систему живой в продакшене	1
Зачем это прикладному инженеру	1
Типовые возражения	2
Итог	2

TL;DR: операционные компоненты держат систему живой в продакшене

За модуль мы разобрали:

- **Прокси и балансировка** — кто стоит впереди приложения, как раздаются запросы между инстансами (Round Robin, Least Connections, Consistent Hashing) и зачем ставить лимит на частоту (Token Bucket, Sliding Window).
- **Безопасность и доступ** — чем отличается аутентификация от авторизации, как RBAC и ABAC выражают права, как живут JWT и OAuth 2.0, и почему в больших компаниях всё сводится к SSO через SAML или OIDC.
- **Observability** — три опоры наблюдаемости: структурированные логи, метрики в Prometheus, распределённые трейсы через OpenTelemetry. Плюс планировщики задач (Celery, Airflow, Dagster), которые отправляют тяжёлую работу в фон.

Зачем это прикладному инженеру

Локально приложение хорошо работает у всех. Боль начинается в проде: трафик внезапно растёт, кто-то ломится в API с тысячей запросов в секунду, пользователь жалуется на ошибку, а её никто не воспроизводит. На эти ситуации отвечают именно операционные компоненты.

Прокси и rate limiting закрывают вход: распределяют трафик, отсекают перегрузки, дают единую точку для SSL и кеша. Без них масштабирование сводится к героическим усилиям, а первый же бот кладёт сервис. Алгоритм балансировки выбирается под форму трафика: одинаковые stateless-инстансы дружат с Round Robin, долгие соединения — с Least Connections, а распределённый кеш живёт на Consistent Hashing, чтобы добавление узла не переразложило все ключи.

Аутентификация и авторизация — два разных вопроса, которые часто склеивают. Первый отвечает «кто ты», второй — «что тебе можно». RBAC хорош для понятных иерархий, ABAC — там, где правила зависят от времени, IP, отдела или статуса MFA. JWT удобен

для stateless-сервисов, но требует продумать отзыв токенов. OAuth и OIDC закрывают сценарии, где сторонним приложениям нужен доступ без передачи пароля.

Observability — то, без чего поддерживать систему в проде невозможно. Логи отвечают на вопрос «что случилось», метрики — «сколько и как быстро», трейсы — «через какие сервисы прошёл запрос и где он завис». Структурированный JSON-лог с `trace_id` склеивает все три уровня в одну картину, а Prometheus + Grafana превращают эту картину в дашборды и алерты.

Типовые возражения

«У нас два инстанса, прокси не нужен» обычно длится ровно до первого инцидента: один процесс падает, и без балансировщика никто не уведёт трафик на живой. Прокси перед приложением окупается даже на двух репликах: SSL termination, единая точка логов, базовая защита от всплесков.

«JWT решает все проблемы с сессиями» — отчасти. Stateless-валидация подписи действительно масштабируется лучше серверных сессий, но логика отзыва (logout, скомпрометированный токен, смена прав) добавляет либо короткий TTL с refresh-токенами, либо blacklist в кеше. Иначе токен действует до `exp`, и поделать с этим нечего.

«Запустим — потом подключим логи и метрики» обычно превращается в отладку по `print` через SSH и нервную пятницу. Метрики и структурированные логи дешевле прикручивать сразу: позже их добавление упирается в рефакторинг каждого хендлера и обязательку трассировки.

Итог

Операционные компоненты не делают бизнес-логику. Они отвечают за то, чтобы приложение выдержало реальный трафик, не пускало посторонних к чужим данным и продолжало работать предсказуемо, когда что-то ломается. Без прокси не масштабируешь, без аутентификации не выпустишь в интернет, без observability не починишь то, чего не видишь.